Technische Hochschule Würzburg-Schweinfurt

Bachelor of Engineering in Robotics

Internship Report

LM450 Autonomous Navigation

Student	Rishabh Rudani
University	THWS
Program	Bachelor of Engineering in Robotics
Semester	5
Company	Lemvos Robotics GmbH
Team	Robotics Software Development for USV
Internship Period	1 October 2025 – 31 March 2026
University Supervisor	Dorit Bormann
Company Supervisors	Simone Contorno and Phillip

Abstract

This report documents my internship work on the LM450 autonomous navigation workspace at Lemvos Robotics GmbH. The project focused on a ROS 2 Humble based autonomy stack for an unmanned surface vessel (USV) and covered localization, offline water-map generation, navigation with Nav2, mission execution through BehaviorTree.CPP, and runtime debugging. My work was distributed across five main authored packages: `automatepro_localization`, `lm450_global_costmap`, `lm450_nav2`, `lm450_waypoints`, and `debugging_lm450_path`. Together these packages form a complete workflow for simulation and real deployment.

The report focuses only on the work performed, the implemented system structure, the validation workflow, and the technical knowledge gained during the internship. The final validation included focused package-level testing as well as integrated navigation testing, and the navigation system completed continuous operation over a full day of testing.

Contents

Abstract **i**

1 Introduction **1**

1.1 Company Context 1

1.2 Department and Team 1

1.3 Internship Objective 1

2 Project Overview **2**

2.1 Scope of the Internship Work 2

2.2 Main Authored Packages 2

2.3 End-to-End Architecture 3

2.4 Runtime Interfaces and Frame Ownership 4

3 Localization and Mapping **5**

3.1 Localization with automatepro_localization 5

3.2 Offline Mapping with lm450_global_costmap 6

4 Navigation and Control **7**

4.1 Navigation Stack with lm450_nav2 7

4.2 Command Normalization and Safety Logic 8

4.3 Simulation and Hardware Tuning 8

5 Perception and Local Obstacle Avoidance **9**

5.1 Water-Obstacle Detection Pipeline 9

5.2 Mask-to-Point-Cloud Conversion 9

5.3 Integration with the Local Costmap 10

5.4 Validation of the Perception-Integrated Navigation Stack 10

6 Mission Execution with lm450_waypoints **11**

6.1 Role of lm450_waypoints in the Full Stack 11

6.2 Repository Structure and Active Architecture 11

6.3 Mission Flow and Mode Handling 12

6.4 Dependencies, Configuration, and Operational Tooling 13

7 Debugging, Testing, and Validation **14**

7.1 Path-Following Debugging 14

7.2 Focused Automated Testing 14

7.3 System-Level Validation Workflow 15

7.4 Repository Metrics for lm450_waypoints 15

8 Repository Organization **16**

8.1 Detailed Structure of lm450_waypoints 16

8.2 Repository Areas and Their Role 18

9 Knowledge and Learning Outcomes **19**

10 Conclusion **20**

A Authored Package Inventory **21**

Chapter 1

Introduction

1.1 Company Context

This report describes my internship at Lemvos Robotics GmbH in the Robotics Software Development for USV team. The industrial context of the internship was the development of the LM450 autonomous navigation stack for an unmanned surface vessel. Unlike a narrowly scoped academic exercise, this work required multiple robotics subsystems to function together as one reliable software pipeline. Localization, map preparation, planning, control, mission supervision, and debugging all had to share consistent interfaces and frame semantics.

1.2 Department and Team

My work was carried out within the Robotics Software Development for USV team. The task environment combined academic expectations with practical engineering constraints. On the academic side, the system had to be understandable in terms of algorithms, architecture, and data flow. On the industrial side, it had to run in both simulation and real deployment, support operator workflows, and be maintainable for repeated validation.

1.3 Internship Objective

The main objective of the internship was to contribute to a complete autonomous navigation workflow for the LM450 vessel. In practical terms, the system had to convert GNSS, IMU, and magnetometer data into a reliable fused pose, use marine-relevant offline maps for traversability, generate and track paths through Nav2, manage mission logic with explicit state transitions, and provide debugging tools for evaluating path-following quality. A further objective was to analyze and document the `lm450_waypoints` package in depth, because it acts as the mission-level bridge between operator intent and the lower-level navigation stack.

Chapter 2

Project Overview

2.1 Scope of the Internship Work

The internship did not focus on only one node or one isolated algorithm. Instead, it covered the integration of estimation, map preparation, planning, control adaptation for a marine platform, mission execution, and post-run debugging. This made the work especially valuable because every engineering decision had to be validated not only in isolation, but also in terms of its effect on the complete runtime behavior of the vessel.

2.2 Main Authored Packages

Table 2.1: Main authored packages in the LM450 workspace

Package	Version	Primary role
`automatepro_localization`	2.0.0	GNSS / IMU fusion, heading estimation, datum handling, and EKF integration
`lm450_global_costmap`	1.1.0	Offline water costmap generation from geographic data
`lm450_nav2`	1.0.0	Nav2 planning, control, launch, and command remapping
`lm450_waypoints`	1.1.0	Behavior-tree mission execution, service interfaces, docking, hold, and safety actions
`debugging_lm450_path`	0.0.0	Path-following evaluation, metrics logging, and runtime debugging

In addition to these authored packages, the workspace also included operator-facing documentation, generated costmap outputs, and tooling around BehaviorTree.CPP and Groot. These supporting elements were important for daily development and validation, even though the central engineering work remained concentrated in the five main packages listed above.

2.3 End-to-End Architecture

Table 2.2: Layered architecture of the LM450 autonomy stack

Layer	Main nodes / tools	Purpose
Sensors	GNSS, IMU, magnetometer	Provide global position, motion, and heading-related data
Localization	`gps_to_pose`, `imu_mag_fuser`, `gps_velocity_estimator`, `ekf_node`	Convert raw sensor data into a consistent fused vehicle pose
Mapping	`generate_water_costmap.py`, water costmap PGM/YAML outputs	Prepare a water-based occupancy map for navigation
Navigation	`map_server`, `planner_server`, `controller_server`, `bt_navigator`, `cmd_vel_remap`	Plan paths and generate usable motion commands for the vessel
Mission	`mission_supervisor`, `ExecuteMissionAction`, `HoldPositionAction`, `UndockAction`, `DockAction`, `SafeStopAction`	Manage waypoint execution, hold behavior, docking, undocking, and emergency transitions
Debugging	`path_follow_debugger`, `split_mcap_metrics.py`	Measure and evaluate path-following performance online and offline

The architecture was intentionally layered so that each subsystem had a clear responsibility. Estimation was kept separate from planning, planning was kept separate from mission logic, and debugging remained outside production control nodes. This separation improved maintainability and also made the system easier to validate during integration.

2.4 Runtime Interfaces and Frame Ownership

Table 2.3: Main runtime interfaces used across the stack

Interface	Type	Purpose
`/localization/gps_pose`	PoseWithCovarianceStamped	GNSS converted into a local metric pose
`/localization/imu/data_corrected`	sensor_msgs/Imu	Corrected IMU orientation after magnetic fusion
`/localization/gps_velocity`	TwistWithCovarianceStamped	Planar velocity estimate derived from GPS pose updates
`/localization/odometry/filtered`	nav_msgs/Odometry	Final EKF output used by the rest of the stack
`/compute_path_to_pose`	Nav2 action	Hold-position replanning
`/follow_path`	Nav2 action	Correction execution for hold behavior
`/cmd_vel`	geometry_msgs/Twist	Raw navigation velocity command

Table 2.4: Frame ownership in the LM450 runtime

Transform	Owner	Meaning
map → odom	`gps_to_pose`	Anchors local odometry to the configured world frame
odom → base_link	`ekf_node`	Publishes the fused vehicle pose used by planner and controller
base_link → sensor frames	Robot model / external TF sources	Represents sensor mounting geometry

A key part of the internship was understanding that autonomy reliability is strongly influenced by frame ownership. If the mapping, localization, and control layers disagree about coordinate meaning, the vessel may still produce outputs, but those outputs cannot be trusted operationally.

Chapter 3

Localization and Mapping

3.1 Localization with automatepro_localization

The `automatepro_localization` package forms the state-estimation layer of the system. Its responsibility is to convert raw GNSS, IMU, and magnetometer streams into a fused pose that is suitable for downstream planning and mission execution. During my internship work, this package was central because it established the geometric consistency on which all later navigation logic depends.

Table 3.1: Core components of automatepro_localization

File / component	Role
`src/gps_to_pose.cpp`	Converts latitude and longitude into a local metric pose and publishes the `map->odom` transform
`src/imu_mag_fuser.cpp`	Fuses gyroscope and magnetometer information to estimate yaw
`src/gps_velocity_estimator.cpp`	Computes planar velocity from GPS pose changes
`launch/localization.launch.py`	Starts the localization package together with the `robot_localization` EKF
`config/config_sim.yaml`	Simulation parameters
`config/config_lm450.yaml`	Hardware parameters

A major design decision in this package is that the first valid GPS fix defines the local odometry datum, while the map frame remains tied to the configured world origin. This gives stable semantics to each frame. Heading estimation uses a lightweight complementary approach in which the gyroscope contributes short-term responsiveness and the magnetometer contributes long-term correction. These corrected signals are then fused by `robot_localization` to generate the filtered odometry consumed by the rest of the stack.

3.2 Offline Mapping with lm450_global_costmap

The `lm450_global_costmap` package prepares the navigation map used by Nav2. Since the platform is a USV, the costmap is not a standard terrestrial occupancy grid. The representation must distinguish traversable water from non-navigable land and preserve shoreline constraints so that path planning remains operationally meaningful.

Table 3.2: Main files and data sources in lm450_global_costmap

File / source	Role
`scripts/generate_water_costmap.py`	Main generator for water-centric occupancy maps
`config/baggersee.yaml`	Real-world operating area definition
`config/vrx.yaml`	Simulation area definition
`launch/lm450_global_costmap`.launch.py	Standalone map bring-up
`requirements.txt`	Python geospatial dependencies

The processing pipeline loads water polygons from shapefiles and OpenStreetMap or OSMnx-derived data, merges and dissolves those geometries, rasterizes the water mask, encodes water as free space, leaves land as unknown or non-navigable, and exports PGM and YAML files for repeated use. This package therefore serves as an offline map-generation tool rather than an online SLAM component.

Chapter 4

Navigation and Control

4.1 Navigation Stack with lm450_nav2

The runtime navigation layer is implemented in `lm450_nav2`. My work in this area involved adapting Nav2 to a marine platform so that general-purpose navigation concepts could be translated into behavior that is appropriate for a real vessel. Boats behave differently from ground robots: they are strongly forward-biased, their turn behavior is constrained by dynamics, and actuation must respect safety as well as propulsion limits.

Table 4.1: Main files in lm450_nav2

File	Role
`launch/lm450_nav2`.launch.py	Main navigation bring-up
`config/config_sim.yaml`	Simulation tuning
`config/config_lm450.yaml`	Hardware tuning
src/`cmd_vel`_remap.cpp	Velocity normalization, clamping, acceleration limiting, and marine safety logic
`src/move_base_goal_bridge.cpp`	Legacy goal bridging into Nav2 actions
`src/smac_planner_expansions.cpp`	Planner debug visualization
`scripts/controller_config.py`	Controller-limit calculation helper

The controller strategy was intentionally marine-specific. The system favors forward-oriented motion, avoids aggressive rotate-to-heading behavior that is often unsuitable on water, uses longer or fixed lookahead settings on hardware, and exposes reverse behavior only under controlled mission-level conditions such as undocking.

4.2 Command Normalization and Safety Logic

A particularly important component of `lm450_nav2` is `cmd_vel_remap`. This node is not only a message-format adapter. It also performs safety-critical shaping of the commanded motion. It clamps linear and angular velocities, limits acceleration and deceleration, blocks reverse motion unless the mission layer explicitly enables undocking, enforces a minimum effective command, and maps physical command values into normalized actuator ranges.

This safety-oriented normalization was important in practice because marine actuation often includes dead zones and hardware-specific command ranges. Without this conversion layer, there would be a larger mismatch between planner output and real vessel response.

4.3 Simulation and Hardware Tuning

Table 4.2: Representative differences between simulation and hardware tuning

Category	Simulation	Hardware	Meaning
Base frame	`lm450/lm450/base_link`	`lm450_bow_link`	Platform-specific TF integration
EKF frequency	5.0 Hz	20.0 Hz	Higher update rate on the real platform
Cruise speed	0.6 m/s	2.0 m/s	Higher usable speed on the vessel
Dead-zone floor	0.2 m/s	0.8 m/s	Hardware propulsion has a larger effective dead zone
Lookahead behavior	Velocity-scaled	Fixed long lookahead	Hardware uses more anticipatory steering
RViz default	Enabled	Disabled	Different deployment assumptions
Real undock	False	True	Hardware enables reverse undock behavior

These differences were not cosmetic configuration changes. They reflect the fact that control settings suitable for simulation are often not sufficient for real deployment. The internship helped me understand how much platform behavior influences controller tuning, safety margins, and frame integration.

Chapter 5

Perception and Local Obstacle Avoidance

5.1 Water-Obstacle Detection Pipeline

In addition to localization, mapping, planning, and mission execution, the integrated LM450 workflow also included a perception path for local obstacle avoidance on water. The purpose of this subsystem was to detect relevant obstacles in front of the vessel, convert those detections into a geometric representation, and feed them into the local navigation stack so that avoidance behavior could be handled by the planner and controller.

The detection stage used a UNet segmentation model for water-obstacle detection. The segmentation output separated obstacle regions from the surrounding water surface in camera images. For runtime deployment, Isaac ROS UNet was used to run the segmentation network and generate the mask that represented the detected obstacle area.

5.2 Mask-to-Point-Cloud Conversion

After the segmentation mask was produced, the masked pixels were passed to the inverse-perspective-mapping pipeline. This node projected the relevant image pixels into 3D coordinates and generated a point cloud representing the detected obstacle region in the robot frame. In practice, this step was important because the local navigation stack does not directly consume image masks for avoidance. It requires a spatial representation that can be fused into the costmap and interpreted by the planner.

Table 5.1: Perception pipeline used for local obstacle avoidance

Stage	Function
Camera image acquisition	Provides the forward scene used for obstacle perception
UNet segmentation model	Detects obstacle regions on the water surface
Isaac ROS UNet	Runs inference and produces the runtime segmentation mask
IPM point-cloud node	Projects masked pixels into 3D obstacle points
Local costmap integration	Inserts the obstacle point cloud into the local obstacle layer
Nav2 local planning	Uses the updated local costmap for object avoidance

5.3 Integration with the Local Costmap

The generated point cloud was fed to the local costmap so that the detected obstacles became part of the local obstacle representation used by Nav2. This closed the perception-to-navigation loop: camera observations were converted into semantic masks, the masks were converted into geometric obstacle points, and those points influenced the local planner for object avoidance.

This integration was especially relevant for a marine platform because the environment can include floating or semi-floating obstacles that are not represented in the offline global water map. The local costmap therefore complemented the offline map by adding short-range dynamic obstacle information derived directly from the perception stack.

5.4 Validation of the Perception-Integrated Navigation Stack

The obstacle-detection and costmap-integration workflow was validated as part of the complete navigation system rather than as an isolated perception demo. During integrated testing, the navigation stack ran continuously over a full day while combining localization, offline water costmaps, mission supervision, navigation control, and local obstacle-avoidance inputs. This demonstrated that the perception-generated point-cloud data could be incorporated into the operational navigation pipeline without breaking the stability of the overall system.

Chapter 6

Mission Execution with lm450_waypoints

6.1 Role of lm450_waypoints in the Full Stack

The `lm450_waypoints` package provides the mission-level orchestration layer of the complete LM450 autonomy system. It does not perform low-level control by itself. Instead, it sits above Nav2 and manages operator-facing mission requests, state transitions, mission progression, hold behavior, docking, undocking, and safe-stop actions. This package was especially important in my work because it is where operator intent becomes structured runtime behavior.

6.2 Repository Structure and Active Architecture

The current implementation is centered on a single lifecycle node named `mission_supervisor`. The active runtime architecture is intentionally compact. Mission loading and common policy logic are handled in `mission_utils` and `gps_utils`, while each operating mode is implemented as a dedicated BehaviorTree action node.

Table 6.1: Main active components of lm450_waypoints

Component	Primary responsibility
`mission_supervisor`	Lifecycle ownership, services, BT creation and ticking, preemption, and mission/state transitions
`mission_utils/gps_utils`	Mission parsing, origin lookup, yaw completion, GPS conversion, and shared helpers
`ExecuteMissionAction`	Sequential mission execution through Nav2 `NavigateToPose`
`HoldPositionAction`	Drift monitoring, recovery path computation, and follow-path correction
`UndockAction`	Reverse-drive undock with optional latch sequencing and GPS-aware stopping
`DockAction`	Execution of a dedicated dock mission using the same sequential goal pattern
`SafeStopAction`	Emergency stop with repeated zero-velocity publication
`IdleAction`	Neutral state behavior when no mission is active

A major architectural improvement visible in the repository is the consolidation of mission logic around one supervisor node and a persistent Behavior Tree. This design keeps the state model explicit and makes mission progression easier to inspect.

6.3 Mission Flow and Mode Handling

The package supports multiple operating modes under one runtime owner. In the normal mission path, the supervisor loads a mission, creates the behavior tree, and ticks the corresponding action node until all waypoints are completed. The mission may either terminate normally or loop depending on the mission configuration.

Hold mode is implemented as an active behavior rather than a passive status flag. The system captures or reuses a target pose, monitors drift by means of TF, and starts a correction cycle only when the drift threshold is exceeded. This design allows the vessel to remain stable around a hold target without constantly generating unnecessary correction commands.

Undock mode produces reverse motion and supports both GPS-based stopping and a fallback time-distance strategy. On LM450 hardware, the undock behavior can also control latch opening and closing through the relevant drive topic. Dock mode reuses the same sequential navigation concept as a dedicated docking mission. Safe-stop mode repeatedly publishes zero-motion commands and halts the active mission branch.

6.4 Dependencies, Configuration, and Operational Tooling

Table 6.2: Key dependencies used by lm450_waypoints

Dependency	Role in the package
`rclcpp,rclcpp_lifecycle,rclcpp_action`	Lifecycle ownership, services, topics, and action clients
`nav2_msgs`	`NavigateToPose`, `ComputePathToPose`, and `FollowPath` actions
`behaviortree_cpp_v3`	Persistent Behavior Tree execution model
`tf2,tf2_ros,tf2_geometry_msgs`	Pose transforms, hold target capture, and drift monitoring
`yaml-cpp`	Mission and configuration parsing
`GeographicLib`	UTM and geodesic coordinate conversion
`automatepro_interfaces`	Latch drive message type for real undock hardware

The configuration layout of the package is clearly separated. Runtime parameters are stored in `config_sim.yaml` and `config_lm450.yaml`, default missions are stored in `mission_sim.yaml` and `mission_lm450.yaml`, dedicated docking missions are stored in `dock_mission_*.yaml`, and the active behavior tree is defined in `supervisor_bt.xml`. This separation made the package easier to test and reason about during the internship.

Operational scripts also played an important role. Service sequencing, docking tests, and undocking latch tests were implemented through shell scripts, which reflects the practical workflow used to exercise the package during development and validation.

Chapter 7

Debugging, Testing, and Validation

7.1 Path-Following Debugging

I created and used `debugging_lm450_path` as a validation package so that path-following behavior could be measured instead of being judged only through visual observation. The package computes closest-point projection on the reference path and derives signed cross-track error, along-track distance, reference yaw, and heading error. The results can be logged to CSV, processed through MCAP workflows, and visualized through RViz markers.

This part of the work was important because it turned navigation validation into a measurable engineering task. Instead of simply observing whether the vessel looked stable, I could inspect tracking quality with explicit geometric metrics.

7.2 Focused Automated Testing

Table 7.1: Focused automated test inventory for lm450_waypoints

Test file	Case count	Focus
`test/test_gps_conversion.cpp`	10	UTM conversion, geodesic fallback, strict cross-zone rejection, and hemisphere handling
`test/test_mission_utils.cpp`	4	Mission request parsing, new YAML schema, legacy YAML schema, and loop default behavior

These automated tests targeted the highest-risk utility logic in the package. GPS conversion and mission parsing are both areas where silent failures can produce major navigation problems at runtime, so these tests were valuable in increasing confidence in the system.

7.3 System-Level Validation Workflow

Beyond focused unit testing, the integrated LM450 stack was also validated through operational workflows that exercised the mission services, state transitions, docking and undocking behavior, and navigation execution chain from localization to control output. Shell scripts such as the service-sequencing workflow, docking tests, and undocking-latch tests were used as practical validation tools during development.

The final integrated validation included extended runtime testing of the complete navigation pipeline. During this test phase, the LM450 navigation system completed continuous navigation operation over a full day. This full-day validation confirmed that localization, offline costmap usage, Nav2 planning, mission supervision, and command normalization could operate together as one stable system over an extended period rather than only in short demonstration runs.

7.4 Repository Metrics for lm450_waypoints

Table 7.2: Selected repository metrics for lm450_waypoints

Metric	Value
Total files	52
Active files (excluding `deprecated/`)	46
Deprecated files	6
Total lines of content	12,317
Active lines of content	7,998
Deprecated lines of content	4,319
Deprecated share of repository lines of code	35.1%
Source files (`.cpp`, `.hpp`, `.py`, `.sh`)	34
Active C++ classes	17
Active C++ structs	4
Total classes across repository	30
Total structs across repository	7
Automated GTest cases	14
Shell scripts	8

These metrics show that the active runtime is relatively compact, while a visible portion of the repository is still occupied by preserved legacy material. During my analysis, the main runtime complexity was concentrated in mission parsing, state coordination, hold-position correction, and hardware-aware undocking behavior.

Chapter 8

Repository Organization

8.1 Detailed Structure of lm450_waypoints

The following repository structure shows how source code, configuration, scripts, tests, and archived material are organized inside `lm450_waypoints`. This structure was useful during my internship because it made it possible to separate runtime code from validation tooling and legacy content.

Listing 8.1: Repository tree of lm450_waypoints

```
lm450_waypoints/
|-- CMakeLists.txt
|-- package.xml
|-- README.md
|-- TECHNICAL_REPORT.md
|-- config/
|   |-- config_lm450.yaml
|   |-- config_sim.yaml
|   |-- mission_lm450.yaml
|   |-- mission_sim.yaml
|   |-- dock_mission_lm450.yaml
|   |-- dock_mission_sim.yaml
|   `-- supervisor_bt.xml
|-- docs/
|   |-- GPS_CONVERSION.md
|   `-- WAYPOINT_MISSION_FLOW.md
|-- include/lm450_waypoints/
|   |-- bt_plugin_registration.hpp
|   |-- dock_action.hpp
|   |-- execute_mission_action.hpp
|   |-- gps_utils.hpp
|   |-- hold_position_action.hpp
|   |-- idle_action.hpp
|   |-- mission_supervisor.hpp
```

```
|   |-- mission_utils.hpp
|   |-- safe_stop_action.hpp
|   `-- undock_action.hpp
|-- launch/
|   |-- lm450_waypoints.launch.py
|   `-- __pycache__/
|-- msg/
|   `-- WaypointMissionFeedback.msg
|-- scripts/
|   |-- run_service_sequence.sh
|   |-- run_service_sequence_lm450.sh
|   |-- docking/
|   |   |-- run_dock_test.sh
|   |   |-- run_dock_test_lm450.sh
|   |   `-- run_dock_test_sim.sh
|   `-- undocking/
|       |-- run_undock_latch_test.sh
|       |-- run_undock_latch_test_lm450.sh
|       `-- run_undock_latch_test_sim.sh
|-- src/
|   |-- bt_plugin_registration.cpp
|   |-- dock_action.cpp
|   |-- execute_mission_action.cpp
|   |-- hold_position_action.cpp
|   |-- idle_action.cpp
|   |-- mission_supervisor.cpp
|   |-- mission_utils.cpp
|   |-- safe_stop_action.cpp
|   `-- undock_action.cpp
|-- srv/
|   `-- StartWaypointMission.srv
|-- test/
|   |-- TEST.md
|   |-- test_gps_conversion.cpp
|   `-- test_mission_utils.cpp
`-- deprecated/
    |-- README.md
    |-- config/mission_bt.xml
    `-- src/
```

8.2 Repository Areas and Their Role

Table 8.1: Functional repository areas in lm450_waypoints

Area	Function in the package
`src/`	Active runtime implementation
`include/`	Public and internal headers for the active runtime
`config/`	Runtime parameters, mission definitions, and supervisor BT layout
`launch/`	Lifecycle launch entry point
`scripts/`	Manual validation and regression-driving utilities
`test/`	Focused unit tests
`docs/`	Mission flow and GPS conversion documentation
`deprecated/`	Archived legacy implementation retained in the repository

Chapter 9

Knowledge and Learning Outcomes

This internship gave me experience with how a real autonomy stack is built, integrated, and validated in practice. The most important learning outcome was understanding that reliable navigation does not come from one isolated algorithm. It emerges from the agreement between estimation, mapping, planning, mission execution, control adaptation, and debugging.

From the localization work, I learned how critical frame semantics and datum handling are for downstream reliability. From the mapping work, I learned how environmental data can be transformed into a navigation-ready costmap for repeated deployment. From the navigation layer, I learned how a generic middleware component such as Nav2 must be tuned and constrained for a specific platform. From the mission layer, I learned how behavior trees can be used to express explicit operational modes such as mission execution, hold, docking, undocking, and emergency stopping. From the debugging workflows, I learned how to evaluate system quality quantitatively through path-following metrics instead of relying only on visual intuition.

Another important learning outcome was software organization. The internship showed me how repository structure, configuration separation, test placement, launch design, and operational scripts all contribute to the usability of a robotics workspace. In an academic setting it is easy to focus only on algorithmic code, but this project demonstrated that documentation, tooling, and reproducible validation are also essential parts of robotics engineering.

Chapter 10

Conclusion

The LM450 internship work resulted in a complete autonomous navigation workflow that combines sensor fusion, offline marine-map generation, Nav2-based planning and control, behavior-tree mission execution, and dedicated debugging utilities. The work was not limited to isolated node development. Instead, it covered the integration of multiple subsystems into one operational runtime for both simulation and hardware deployment.

The combined analysis of the workspace and the `lm450_waypoints` package shows how mission supervision connects operator requests with the rest of the navigation stack. The completed testing workflow, including focused automated tests and full-day integrated navigation validation, demonstrated that the system could be exercised as a coherent end-to-end autonomy pipeline. This internship therefore provided both practical implementation experience and a deeper technical understanding of how full robotics systems are engineered for real deployment.

Appendix A

Authored Package Inventory

Table A.1: Summary of the main authored packages

Package	Summary
`automatepro_localization`	GNSS / IMU fusion, heading estimation, datum handling, and EKF integration
`lm450_global_costmap`	Offline water-map generation and export for repeated navigation use
`lm450_nav2`	Nav2 integration, marine controller tuning, command normalization, and safety logic
`lm450_waypoints`	Mission execution, docking, undocking, hold behavior, and operator-facing service interfaces
`debugging_lm450_path`	Path-following evaluation, metrics logging, and RViz debugging support